

PINE 2기 · 7회차

멀티 에이전트 · 메모리 · RAG

여러 AI를 연결하고, 기억하게 하고, 검색까지 시킨다

유스AI프로젝트:D | Microsoft Agent Framework · APIM Foundry Proxy

오늘의 여정 · 90분



1부 · 개념 + 코드 데모

30분

3분

복습 브릿지 — 6회차 → 오늘

8분

멀티 에이전트 3패턴 + 라이브 데모

7분

Session 메모리 (기억하는 AI)

10분

RAG — 오늘의 핵심 ★

2분

숙제 & Copilot 안내



2부 · 숙제 실습

60분

AI 학교 생활 도우미 만들기

GitHub Copilot과 함께 스스로 코드를 완성!

- RAG로 우리 학교 정보 검색
- 질문 3개 답변 테스트
- 보너스 🌐 Gradio 웹 UI

6회차에서 배운 것 → 오늘 하는 것



6회차

작가·편집자를 손으로 이어붙임



오늘

SequentialBuilder로 자동 파이프라인



6회차

WorkflowBuilder 첫 경험



오늘

Sequential·Concurrent·GroupChat 전용 빌더



6회차

tool() 도구 등록 · ReAct



오늘

@tool로 검색 도구 → RAG에 연결



6회차

꼬맨틀: 임베딩으로 단어 유사도



오늘

RAG: 임베딩으로 질문↔문서 유사도



핵심 다리: 꼬맨틀은 ‘단어 유사도’, RAG는 ‘문서 유사도’ — 똑같은 임베딩이에요!

에이전트 여러 개를 협업시키는 3가지 방법



Sequential

순차 파이프라인

$A \rightarrow B \rightarrow C$
앞 결과를 뒤로 전달

예) 작가→편집자→검토자



Concurrent

동시 병렬

$A \cdot B \cdot C$ 동시 실행
각자 다른 관점 → 합산

예) 기술·시장·윤리 분석



GroupChat

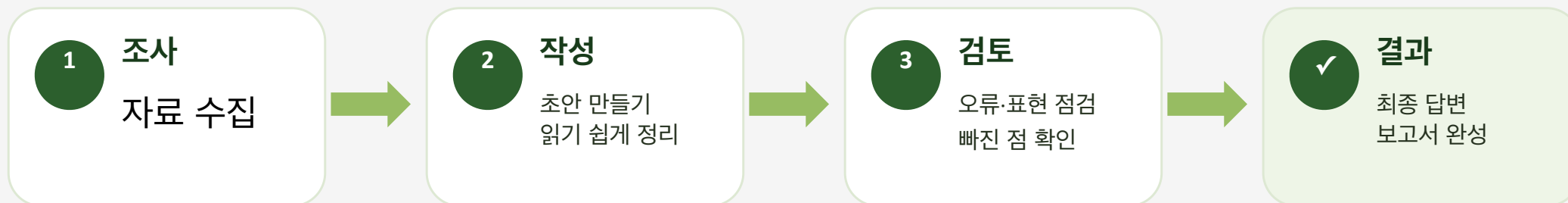
역할 토론

서로 발언을 듣고 대화
누가 말할지 선택

예) 찬성·반대·중재 토론

Sequential: 컨베이어벨트처럼 차례대로

앞 단계의 결과를 다음 단계가 이어받아 완성도를 높입니다.



학교 예시

“진로 탐색 보고서를 만들어줘”
조사 → 작성 → 검토 순서로 처리

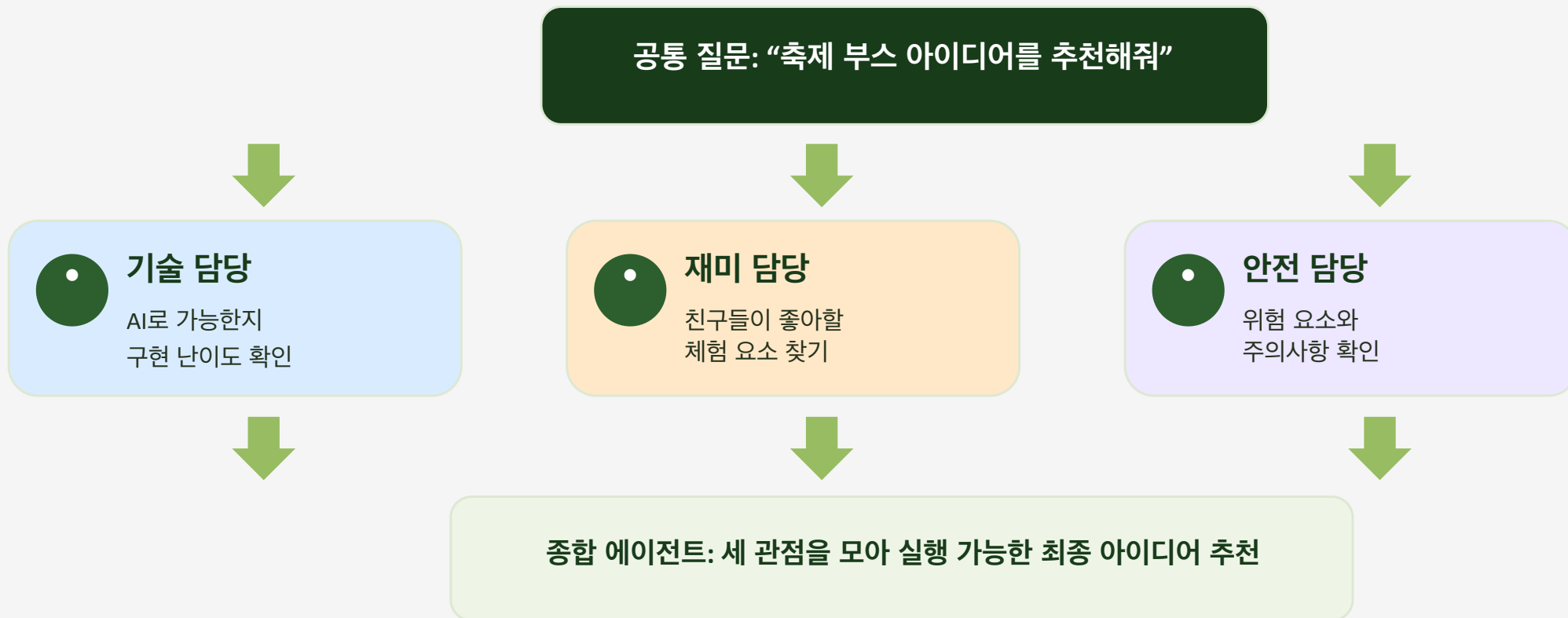
언제 쓰면 좋을까?

순서가 중요한 일
단계가 진행될수록 결과물이 좋아지는 작업

한 줄 기억: “앞 사람이 넘긴 결과를 다음 사람이 이어받는다.”

Concurrent: 조별 과제처럼 동시에 나눠서

서로 기다릴 필요 없는 일을 병렬로 처리하고 마지막에 합칩니다.



한 줄 기억: "같은 질문을 여러 관점으로 동시에 조사한다."

GroupChat: AI 회의실에서 토론하기

여러 에이전트가 서로 의견을 듣고, 반박하고, 합의점을 찾습니다.

토론 주제: 수학여행 장소 고르기



예산 담당

비용이 너무 높으면
안 된다고 주장



재미 담당

친구들이 좋아할
활동을 제안



안전 담당

이동 거리와
안전을 확인



사회자 AI

의견을 정리하고
최종 결론 도출

언제 쓰면 좋을까?

정답이 하나가 아니라 토론·합의·아이디어 발전이 중요한 문제

한 줄 기억: “여러 AI가 회의하며 서로의 생각을 고친다.”

지난 시간엔 손으로, 오늘은 자동으로



6회차 · 수동 (.run 여러 번)

```
draft    = await writer.run(topic)
feedback= await editor.run(draft.text)
final    = await writer.run(
    draft + feedback.text)
# 단계마다 직접 이어붙여야 함 😓
```



7회차 · 자동 (SequentialBuilder)

```
flow = SequentialBuilder(
    participants=[writer, editor, reviewer]
).build()

await flow.run(topic) # 한 줄로 끝! ✨
```

👉 같은 작가·편집자인데, 연결을 프레임워크가 대신 해줍니다.

AI가 이전 대화를 기억하게 하기



세션 없이



“나는 지민이야”



“안녕 지민!”



“내 이름이 뭐야?”



“모르겠어요...” 😓



세션 있어



“나는 지민이야”



“안녕 지민!”



“내 이름이 뭐야?”



“지민이라고 했죠!” ✓

```
session = agent.create_session() # await 없음! (동기 함수)
await agent.run(msg, session=session) # 대화 이력 유지
```

검색(Retrieve) + 생성(Generate) = 오픈북 시험 📖



질문

“동아리 언제 해?”



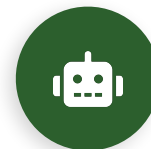
검색

관련 문서 찾기



참고

학교 정보 문서



답변

문서 근거로 생성

일반 LLM

“그 정보는 잘 모르겠어요...”

RAG 에이전트

“동아리는 수요일 4시, 컴퓨터실이에요!” ✓

RAG의 심장 = 6회차에 쓴 그 임베딩



6회차 · 꼬맨틀

단어 ↔ 단어 유사도

“사과” ↔ “바나나”
→ 유사도 0.71

```
cos_sim(vec(w1), vec(w2))
```



7회차 · RAG

질문 ↔ 문서 유사도

“급식 뭐야?” ↔ 학교문서들
→ 가장 비슷한 문서 Top-3

```
cos_sim(vec(질문), vec(문서))
```

👉 대상만 ‘단어→문서’로 바뀌었을 뿐, 완전히 똑같은 원리!

@tool 하나면 에이전트가 알아서 검색한다

```
@tool(approval_mode="never_require")
def search_school_info(query: str) -> str:
    """학교 급식·동아리·규정 정보를 검색한다."""
    q = embed_text(query)
    sims = [cos_sim(q, d) for d in DOCS]
    return top3(sims)    # 가장 비슷한 문서

agent = Agent(client, tools=[search_school_info])
```



포인트

함수 설명(docstring)

이 글이 에이전트에게 '도구 설명서'가 됩니다

자동 호출

질문이 오면 에이전트가 스스로 도구를 부릅니다

근거 있는 답변

찾은 문서에 기반해 답 → 환각(거짓말) ↓

YOUR MISSION · 60분

숙제: AI 학교 생활 도우미 🏫

✓ 필수

- 학교 데이터 10개 이상 채우기
- 임베딩 + @tool로 검색 구현
- 질문 3개 이상 답변 테스트

★ 보너스: Gradio 웹 UI · Session 기억 · 멀티 에이전트

🤖 Copilot 200% 쓰기

주석으로 원하는 걸 한국어로 쓰기

→ Copilot이 코드 제안

막히면 Chat에 “왜 에러 나?” 물어보기

제안은 읽고 이해한 뒤 채택하기!